

Turn your terminal into a collaborative workspace for AI agents

Andrew Young
Automate Capture Research

April 6, 2026

Abstract

The proliferation of autonomous AI agents necessitates robust mechanisms for inter-agent coordination, particularly when agents require shared, stateful interaction environments. Traditional agent architectures rely on abstract message passing, which abstracts away the low-level operational context. AgentShell addresses this gap by transforming the standard terminal interface into a real-time, collaborative workspace. We introduce a system capable of multiplexing multiple autonomous agents within a single pseudo-terminal (PTY) session. The core innovation lies in the `terminal_coordinator` module, which employs Copy-On-Write (COW) virtual terminal buffers synchronized using Operational Transforms (OT) and Vector Clocks. This mechanism ensures strong consistency across concurrent command execution, file editing, and I/O streams generated by disparate agents. This paper details the architecture, the OT-based conflict resolution strategy for PTY operations, and the implementation structure, demonstrating a novel paradigm for shared, stateful AI collaboration.

1 Introduction

The development of sophisticated, multi-agent AI systems is rapidly advancing. These agents are increasingly tasked with complex, multi-step workflows that require interacting with external, stateful environments, such as a command-line interface (CLI) or a shared file system. While existing frameworks excel at high-level task decomposition via message queues or API calls, they often fail to model the fine-grained, sequential, and stateful nature of interactive terminal sessions. When multiple agents must concurrently execute commands, modify files, or observe the same terminal output, maintaining a consistent, shared view of the session state becomes a significant challenge.

The problem addressed by AgentShell is the lack of a mechanism to allow multiple autonomous entities to operate *within* a single, shared, real-time terminal context without corrupting the session state. If Agent A types a command and Agent B simultaneously attempts to edit a file displayed in the terminal, the resulting state must be deterministically resolved.

Our motivation is to bridge the gap between abstract agent orchestration and concrete, stateful environmental interaction. By treating the terminal buffer as a shared, mutable document, we can apply established distributed systems techniques—specifically Operational Transforms—to manage concurrent edits and I/O operations, thereby enabling true collaborative terminal workspaces for AI agents.

2 Related Work

Research in multi-agent systems (MAS) typically focuses on coordination protocols, such as Contract Net or auction-based mechanisms [?]. These approaches manage *intent* but not *state*.

For shared document editing, the field of Collaborative Text Editing (CTE) provides relevant precedents. Systems like Google Docs rely heavily on Operational Transformation (OT) or Conflict-Free Replicated Data Types (CRDTs) to merge concurrent edits deterministically. These techniques are proven for text streams, but applying them to the complex, asynchronous, and structured nature of PTY I/O streams presents unique challenges.

In the context of AI interaction, tools like LangChain or AutoGen facilitate agent communication via structured APIs. However, these frameworks abstract the terminal layer entirely. Our work differs by operating *at* the terminal layer, treating the terminal buffer itself as the shared data structure requiring synchronization.

3 Methodology

The AgentShell architecture is centered around the `terminal_coordinator`, which acts as the authoritative state manager for the shared PTY session.

3.1 Architecture Overview

The system comprises three main components:

1. **PTY Multiplexer:** Manages the underlying operating system pseudo-terminal, routing input/output streams.
2. **Virtual Terminal Buffers (COW):** Each agent maintains a private, mutable view of the terminal state, initialized from the shared buffer. Copy-On-Write ensures that an agent’s local modifications do not immediately affect other agents until a transformation is applied.
3. **Synchronization Engine (OT/Vector Clocks):** This engine receives operations (keystrokes, command outputs, file writes) from agents, transforms them against pending operations from other agents, and commits the resulting canonical operation to the shared state.

3.2 Operational Transforms for PTY Operations

Standard OT is designed for sequential text edits. In AgentShell, we extend this concept to encompass three types of operations ($\mathcal{O}$):

- $\mathcal{O}_{\text{Input}}$: Agent-generated keystrokes or commands.
- $\mathcal{O}_{\text{Output}}$: Terminal output received from the shell process.
- $\mathcal{O}_{\text{File}}$: Atomic file system modifications initiated by an agent.

When Agent A generates an operation O_A based on state S , and Agent B concurrently generates O_B based on S , the coordinator must transform O_A against O_B (resulting in O'_A) and O_B against O_A (resulting in O'_B) such that applying O'_A then O'_B yields the same final state as applying O_A then O_B in a sequential execution.

We utilize Vector Clocks ($\vec{V}$) attached to every operation to establish causal ordering. If $\vec{V}(O_A) < \vec{V}(O_B)$, O_A is causally prior to O_B , and no transformation is necessary beyond simple application. If they are concurrent, the OT algorithm is invoked.

3.3 State Consistency via `session_manager`

The `session_manager` orchestrates the flow. It maintains the canonical state and a log of all committed operations. When an agent requests an action, the manager checks the agent’s local state against the canonical state using the vector clock. If divergence is detected, the agent’s local buffer is rebased by applying the necessary transformations derived from the log.

4 Implementation

The system is modularized across several Python components.

4.1 Core Modules

- `types.py`: Defines the canonical data structures, including the `Operation` object (containing type, payload, and vector clock) and the `VirtualBuffer` state.
- `capability_registry.py`: Manages the capabilities of connected agents (e.g., read-only, write access to specific files) to enforce security and scope.
- `session_manager.py`: The central state machine. It receives raw operations, manages the transformation queue, and commits the resulting canonical operations to the shared buffer.
- `terminal_coordinator`: This module wraps the PTY interface. It intercepts raw I/O, translates it into structured `O` objects, and feeds them into the `session_manager` for synchronization.

4.2 Testing and Verification

Unit tests, defined in `tests/`, focus heavily on the OT logic. Specifically, tests verify that concurrent inputs (e.g., two agents typing at the same character index) resolve deterministically to a single, predictable final buffer state, confirming the correctness of the transformation functions.

5 Results and Discussion

Initial simulations demonstrate that AgentShell successfully maintains state consistency under high concurrency. When two agents simultaneously issue commands that result in overlapping terminal output, the system resolves the conflict by applying the operations in a causally consistent order, ensuring that the final displayed output reflects both actions without corruption.

The use of COW buffers is critical for performance. Agents operate locally on their private copies, incurring minimal overhead until a conflict necessitates a transformation and subsequent state merge. The overhead is dominated by the complexity of the OT algorithm, which is $O(N)$, where N is the number of operations in the conflicting history, which is acceptable for typical interactive session lengths.

6 Conclusion and Future Work

AgentShell provides a novel framework for enabling truly collaborative, stateful interaction among autonomous AI agents within a shared terminal environment. By integrating Operational Transforms with PTY management, we solve the problem of concurrent state mutation in a shared CLI context.

Future work will focus on extending the OT framework to handle non-textual, structured operations, such as complex shell piping or environment variable modifications, which require richer semantic transformation rules beyond simple character insertion/deletion. Furthermore, integrating a formal verification layer for the transformation logic is planned.